

CS322: Neural Network

Vector Calculus

Sachchida Nand Chaurasia
Assistant Professor

Department of Computer Science
Banaras Hindu University
Varanasi

Email id: snchaurasia@bhu.ac.in, sachchidanand.mca07@gmail.com



March 24, 2025

Outline

Partial derivative

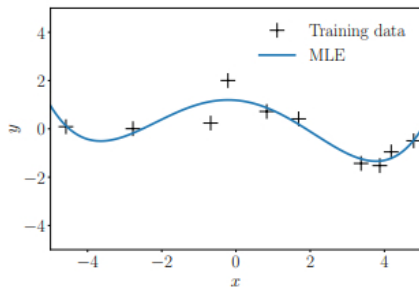
Gradients of Vector-Valued Functions

Gradient Descent and it's Variants

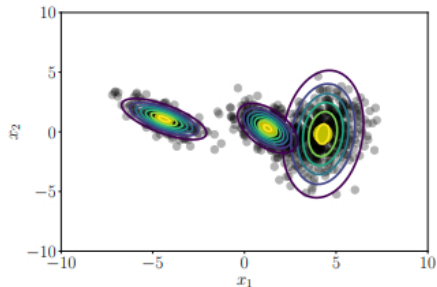
Vector Calculus I

- Many algorithms in machine learning optimize an objective function with respect to a set of desired model parameters that control how well a model explains the data: Finding good parameters can be phrased as an optimization problem.
- Curve-fitting problems and optimize linear weight parameters to maximize the likelihood
- Neural-network auto-encoders for dimensionality reduction and data compression, where the parameters are the weights and biases of each layer, and where we minimize a reconstruction error by repeated application of the chain rule
- Gaussian mixture models for modeling data distributions, where we optimize the location and shape parameters of each mixture component to maximize the likelihood of the model.

Vector Calculus II



(a) Regression problem: Find parameters, such that the curve explains the observations (crosses) well.



(b) Density estimation with a Gaussian mixture model: Find means and covariances, such that the data (dots) can be explained well.

Outline

Partial derivative

Gradients of Vector-Valued Functions

Gradient Descent and it's Variants

Partial derivative I

- Differentiation applies to functions f of scalar variable $x \in \mathbb{R}$.
- When a function f depends on one or more variables $x \in \mathbb{R}^n$, e.g., $f(x) = f(x_1, x_2)$.
- The generalization of the derivative to functions of several variables is the *gradient*.
- We find the gradient of the function f with respect to x by varying one variable at a time and keeping the others constant.
- The gradient is then the collection of these *partial derivatives*.

Partial derivative II

Partial Derivative: For a function $f : \mathbb{R}^n \rightarrow \mathbb{R}$, $\mathbf{x} \mapsto f(\mathbf{x})$ $\mathbf{x} \in \mathbb{R}^n$ of n variables $x_1, \dots, x_n$.

$$\frac{\partial f}{\partial x_1} = \lim_{h \rightarrow 0} \frac{f(x_1 + h, \dots, x_n) - f(\mathbf{x})}{h} \quad (1)$$

$$\frac{\partial f}{\partial x_2} = \lim_{h \rightarrow 0} \frac{f(x_1, x_2 + h, \dots, x_n) - f(\mathbf{x})}{h} \quad (2)$$

$$\vdots \quad (3)$$

$$\frac{\partial f}{\partial x_n} = \lim_{h \rightarrow 0} \frac{f(x_1, \dots, x_{n-1}, x_n + h) - f(\mathbf{x})}{h} \quad (4)$$

✓ The collection of partial derivatives in the row vector

Partial derivative III

$$\nabla_{\mathbf{x}} f = \text{grad } f = \frac{df}{d\mathbf{x}} = \left[\frac{\partial f(\mathbf{x})}{\partial x_1} \quad \frac{\partial f(\mathbf{x})}{\partial x_2} \quad \cdots \quad \frac{\partial f(\mathbf{x})}{\partial x_n} \right] \in \mathbb{R}^{1 \times n}, \quad (5)$$

where n is the number of variables and 1 is the dimension of the image/range/codomain of f .

✓ The row vector is called *gradient* of f or the *Jacobian* and is the generalization of the derivative.

Example: Partial derivatives of $f(x, y) = (x + 2y^3)^2$ using chain rule.

$$\frac{\partial f(x, y)}{\partial x} = 2(x + 2y^3) \frac{\partial}{\partial x}(x + 2y^3) = 2(x + 2y^3)$$

$$\frac{\partial f(x, y)}{\partial y} = 2(x + 2y^3) \frac{\partial}{\partial y}(x + 2y^3) = 12(x + 2y^3)y^2$$

Partial derivative IV

Example: For $f(x_1, x_2) = x_1^2 x_2 + x_1 x_2^3 \in \mathbb{R}$, the partial derivatives of with respect to x_1 and x_2 are

$$\frac{\partial f(x_1, x_2)}{\partial x_1} = 2x_1 x_2 + x_2^3$$

$$\frac{\partial f(x_1, x_2)}{\partial x_2} = x_1^2 + 3x_1 x_2^2$$

✓ The gradient is

$$\frac{df}{d\mathbf{x}} = \left[\frac{\partial f(x_1, x_2)}{\partial x_1} \quad \frac{\partial f(x_1, x_2)}{\partial x_2} \right] = [2x_1 x_2 + x_2^3 \quad x_1^2 + 3x_1 x_2^2] \in \mathbb{R}^{1 \times 2}$$

Partial derivative V

Partial Differentiation Rules:

$$\text{Product rule : } \frac{\partial}{\partial \mathbf{x}} (f(\mathbf{x})g(\mathbf{x})) = \frac{\partial f}{\partial \mathbf{x}} g(\mathbf{x}) + f(\mathbf{x}) \frac{\partial g}{\partial \mathbf{x}} \quad (6)$$

$$\text{Sum rule : } \frac{\partial}{\partial \mathbf{x}} (f(\mathbf{x}) + g(\mathbf{x})) = \frac{\partial f}{\partial \mathbf{x}} + \frac{\partial g}{\partial \mathbf{x}} \quad (7)$$

$$\text{Chain rule : } \frac{\partial}{\partial \mathbf{x}} (g \circ f)(\mathbf{x}) = \frac{\partial}{\partial \mathbf{x}} (g(f(\mathbf{x}))) = \frac{\partial g}{\partial f} \frac{\partial f}{\partial \mathbf{x}} \quad (8)$$

Outline

Partial derivative

Gradients of Vector-Valued Functions

Gradient Descent and it's Variants

Gradients of Vector-Valued Functions I

- ✓ Partial derivatives and gradients of functions $f : \mathbb{R} \rightarrow \mathbb{R}$ mapping to the real numbers.
- ✓ Partial derivatives and gradient to vector-valued functions(vector fields) $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^m$, where $n \geq 1$ and $m > 1$.
- ✓ For a function $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^m$ and a vector $\mathbf{x} = [x_1, \dots, x_n]^T \in \mathbb{R}^n$, the corresponding vector of function values is given as

$$\mathbf{f}(\mathbf{x}) = \begin{bmatrix} f_1(\mathbf{x}) \\ \vdots \\ f_m(\mathbf{x}) \end{bmatrix} \in \mathbb{R}^m \quad (9)$$

- ✓ The partial derivative of a vector-valued function $\mathbf{f} : \mathbb{R}^n \rightarrow \mathbb{R}^m$ with respect to $x_i \in \mathbb{R}$, $i = 1, \dots, n$, is given as the vector

Gradients of Vector-Valued Functions II

$$\frac{\partial \mathbf{f}}{\partial x_i} = \begin{bmatrix} \frac{\partial f_1}{\partial x_i} \\ \vdots \\ \frac{\partial f_m}{\partial x_i} \end{bmatrix} = \begin{bmatrix} \lim_{h \rightarrow 0} \frac{f_1(x_1, \dots, x_{i-1}, x_i+h, x_{i+1}, \dots, x_n) - f_1(\mathbf{x})}{h} \\ \vdots \\ \lim_{h \rightarrow 0} \frac{f_m(x_1, \dots, x_{i-1}, x_i+h, x_{i+1}, \dots, x_n) - f_m(\mathbf{x})}{h} \end{bmatrix} \in \mathbb{R}^m.$$

✓ The gradient of $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ with respect to $\mathbf{x} \in \mathbb{R}^n$ by collecting these partial derivatives:

$$\begin{aligned} \frac{d\mathbf{f}(\mathbf{x})}{d\mathbf{x}} &= \begin{bmatrix} \boxed{\frac{\partial \mathbf{f}(\mathbf{x})}{\partial x_1}} \cdots \boxed{\frac{\partial \mathbf{f}(\mathbf{x})}{\partial x_n}} \end{bmatrix} \\ &= \begin{bmatrix} \boxed{\frac{\partial f_1(\mathbf{x})}{\partial x_1}} \cdots \boxed{\frac{\partial f_1(\mathbf{x})}{\partial x_n}} \\ \vdots \\ \boxed{\frac{\partial f_m(\mathbf{x})}{\partial x_1}} \cdots \boxed{\frac{\partial f_m(\mathbf{x})}{\partial x_n}} \end{bmatrix} \in \mathbb{R}^{m \times n} \end{aligned}$$

Gradients of Vector-Valued Functions III

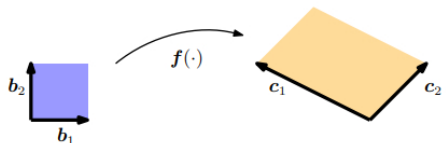
The collection of all first-order partial derivatives of a vector-valued function $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ is called the Jacobian.

✓ The Jacobian $\mathcal{J}$ is an $m \times n$ matrix is as follows:

$$\begin{aligned} \mathbf{J} &= \nabla_{\mathbf{x}} \mathbf{f} = \frac{d\mathbf{f}(\mathbf{x})}{d\mathbf{x}} = \begin{bmatrix} \frac{\partial \mathbf{f}(\mathbf{x})}{\partial x_1} & \cdots & \frac{\partial \mathbf{f}(\mathbf{x})}{\partial x_n} \end{bmatrix} \\ &= \begin{bmatrix} \frac{\partial f_1(\mathbf{x})}{\partial x_1} & \cdots & \frac{\partial f_1(\mathbf{x})}{\partial x_n} \\ \vdots & & \vdots \\ \frac{\partial f_m(\mathbf{x})}{\partial x_1} & \cdots & \frac{\partial f_m(\mathbf{x})}{\partial x_n} \end{bmatrix}, \\ \mathbf{x} &= \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix}, \quad J(i, j) = \frac{\partial f_i}{\partial x_j}. \end{aligned}$$

Gradients of Vector-Valued Functions IV

Jacobian of f can be used to compute the magnifier between the blue and orange area.



✓ Given two vectors $\mathbf{b}_1 = [1, 0]^T$ and $\mathbf{b}_2 = [0, 1]^T$, the area of this square is

$$\left| \det \left(\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \right) \right| = 1.$$

✓ Given two vectors $\mathbf{c}_1 = [-2, 1]^T$ and $\mathbf{c}_2 = [1, 1]^T$, the area of this parallelogram is

Gradients of Vector-Valued Functions V

$$\left| \det \left(\begin{bmatrix} -2 & 1 \\ 1 & 1 \end{bmatrix} \right) \right| = |-3| = 3,$$

- ✓ The area of parallelogram is exactly three times the area of the unit square.
- ✓ A scaling factor can be found by finding a mapping that transforms the unit square into the other square.
- ✓ A variable transformation from $(\mathbf{b}_1, \mathbf{b}_2)$ to $(\mathbf{c}_1, \mathbf{c}_2)$ can be done in linear algebra.

Partial Derivatives Approach: For this approach, we consider a function $f : \mathbb{R}^2 \rightarrow \mathbb{R}^2$ that performs a variable transformation.

- ✓ f maps the coordinate representation of any vector $\mathbf{x} \in \mathbb{R}^2$ with respect to $(\mathbf{b}_1, \mathbf{b}_2)$ onto the coordinate representation $\mathbf{y} \in \mathbb{R}^2$ with respect to $(\mathbf{c}_1, \mathbf{c}_2)$.
- ✓ Identifying the mapping so that we can compute how an area (or volume) changes when it is being transformed by f . For this we need to find out how $f(\mathbf{x})$ changes if we modify $\mathbf{x}$ a bit.
- ✓ This question is exactly answered by the Jacobian matrix $\frac{d\mathbf{f}}{d\mathbf{x}} \in \mathbb{R}^{2 \times 2}$.

Gradients of Vector-Valued Functions VI

$$y_1 = -2x_1 + x_2 \quad (10)$$

$$y_2 = x_1 + x_2 \quad (11)$$

We obtain the function relationship between $\mathbf{x}$ and $\mathbf{y}$, which allows us to get the partial derivatives

$$\frac{\partial y_1}{\partial x_1} = -2, \quad \frac{\partial y_1}{\partial x_2} = 1, \quad \frac{\partial y_2}{\partial x_1} = 1, \quad \frac{\partial y_2}{\partial x_2} = 1$$

and compose the Jacobian as

Gradients of Vector-Valued Functions VII

$$\mathbf{J} = \begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \frac{\partial y_1}{\partial x_2} \\ \frac{\partial y_2}{\partial x_1} & \frac{\partial y_2}{\partial x_2} \end{bmatrix} = \begin{bmatrix} -2 & 1 \\ 1 & 1 \end{bmatrix} .$$

- ✓ The Jacobian represents the coordinate transformation we are looking for.

Gradients of Vector-Valued Functions VIII

Example: Chain rule

Consider the function $h : \mathbb{R} \rightarrow \mathbb{R}$, $h(t) = (f \circ g)(t)$ with

$$f : \mathbb{R}^2 \rightarrow \mathbb{R}$$

$$g : \mathbb{R} \rightarrow \mathbb{R}^2$$

$$f(\mathbf{x}) = \exp(x_1 x_2^2),$$

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = g(t) = \begin{bmatrix} t \cos t \\ t \sin t \end{bmatrix}$$

and compute the gradient of h with respect to t . Since $f : \mathbb{R}^2 \rightarrow \mathbb{R}$ and $g : \mathbb{R} \rightarrow \mathbb{R}^2$ we note that

$$\frac{\partial f}{\partial \mathbf{x}} \in \mathbb{R}^{1 \times 2}, \quad \frac{\partial g}{\partial t} \in \mathbb{R}^{2 \times 1}.$$

The desired gradient is computed by applying the chain rule:

$$\begin{aligned} \frac{dh}{dt} &= \frac{\partial f}{\partial \mathbf{x}} \frac{\partial \mathbf{x}}{\partial t} = \begin{bmatrix} \frac{\partial f}{\partial x_1} & \frac{\partial f}{\partial x_2} \end{bmatrix} \begin{bmatrix} \frac{\partial x_1}{\partial t} \\ \frac{\partial x_2}{\partial t} \end{bmatrix} \\ &= \begin{bmatrix} \exp(x_1 x_2^2) x_2^2 & 2 \exp(x_1 x_2^2) x_1 x_2 \end{bmatrix} \begin{bmatrix} \cos t - t \sin t \\ \sin t + t \cos t \end{bmatrix} \\ &= \exp(x_1 x_2^2) (x_2^2 (\cos t - t \sin t) + 2 x_1 x_2 (\sin t + t \cos t)), \end{aligned}$$

where $x_1 = t \cos t$ and $x_2 = t \sin t$; see (5.72).

Gradients of Vector-Valued Functions IX

Gradient of Vectors with Respect to Matrices:

- We will encounter situations where we need to take gradients of matrices with respect to vectors (or other matrices), which results in a multidimensional tensor.
- We can think of this tensor as a multidimensional array that collects partial derivatives. For example, if we compute the gradient of an $m \times n$ matrix $\mathbf{A}$ with respect to a $p \times p$ matrix $\mathbf{B}$.
- The resulting Jacobian would be $(m \times n) \times (p \times p)$, i.e., a four dimensional tensor $\mathcal{J}$, whose entries are given as $\mathcal{J}_{ijkl} = \partial A_{ij} / \partial B_{kl}$.

Gradients of Vector-Valued Functions X

Let us consider the following example, where

$$\mathbf{f} = \mathbf{A}\mathbf{x}, \quad \mathbf{f} \in \mathbb{R}^M, \quad \mathbf{A} \in \mathbb{R}^{M \times N}, \quad \mathbf{x} \in \mathbb{R}^N \quad (5.85)$$

and where we seek the gradient $d\mathbf{f}/d\mathbf{A}$. Let us start again by determining the dimension of the gradient as

$$\frac{d\mathbf{f}}{d\mathbf{A}} \in \mathbb{R}^{M \times (M \times N)}. \quad (5.86)$$

By definition, the gradient is the collection of the partial derivatives:

$$\frac{d\mathbf{f}}{d\mathbf{A}} = \begin{bmatrix} \frac{\partial f_1}{\partial \mathbf{A}} \\ \vdots \\ \frac{\partial f_M}{\partial \mathbf{A}} \end{bmatrix}, \quad \frac{\partial f_i}{\partial \mathbf{A}} \in \mathbb{R}^{1 \times (M \times N)}. \quad (5.87)$$

To compute the partial derivatives, it will be helpful to explicitly write out the matrix vector multiplication:

$$f_i = \sum_{j=1}^N A_{ij}x_j, \quad i = 1, \dots, M, \quad (5.88)$$

and the partial derivatives are then given as

$$\frac{\partial f_i}{\partial A_{iq}} = x_q. \quad (5.89)$$

This allows us to compute the partial derivatives of f_i with respect to a row of $\mathbf{A}$, which is given as

$$\frac{\partial f_i}{\partial A_{i,:}} = \mathbf{x}^\top \in \mathbb{R}^{1 \times N}, \quad (5.90)$$

Gradients of Vector-Valued Functions XI

$$\frac{\partial f_i}{\partial \mathbf{A}_{k \neq i,:}} = \mathbf{0}^\top \in \mathbb{R}^{1 \times 1 \times N} \quad (5.91)$$

where we have to pay attention to the correct dimensionality. Since f_i maps onto $\mathbb{R}$ and each row of $\mathbf{A}$ is of size $1 \times N$, we obtain a $1 \times 1 \times N$ -sized tensor as the partial derivative of f_i with respect to a row of $\mathbf{A}$.

We stack the partial derivatives (5.91) and get the desired gradient in (5.87) via

$$\frac{\partial f_i}{\partial \mathbf{A}} = \begin{bmatrix} \mathbf{0}^\top \\ \vdots \\ \mathbf{0}^\top \\ \mathbf{x}^\top \\ \mathbf{0}^\top \\ \vdots \\ \mathbf{0}^\top \end{bmatrix} \in \mathbb{R}^{1 \times (M \times N)}. \quad (5.92)$$

Gradients of Vector-Valued Functions XII

Gradient of Matrices with Respect to Matrices:

Consider a matrix $\mathbf{R} \in \mathbb{R}^{M \times N}$ and $\mathbf{f} : \mathbb{R}^{M \times N} \rightarrow \mathbb{R}^{N \times N}$ with

$$\mathbf{f}(\mathbf{R}) = \mathbf{R}^\top \mathbf{R} =: \mathbf{K} \in \mathbb{R}^{N \times N}, \quad (5.93)$$

where we seek the gradient $d\mathbf{K}/d\mathbf{R}$.

To solve this hard problem, let us first write down what we already know: The gradient has the dimensions

$$\frac{d\mathbf{K}}{d\mathbf{R}} \in \mathbb{R}^{(N \times N) \times (M \times N)}, \quad (5.94)$$

which is a tensor. Moreover,

$$\frac{dK_{pq}}{d\mathbf{R}} \in \mathbb{R}^{1 \times M \times N} \quad (5.95)$$

for $p, q = 1, \dots, N$, where K_{pq} is the (p, q) th entry of $\mathbf{K} = \mathbf{f}(\mathbf{R})$. Denoting the i th column of $\mathbf{R}$ by $\mathbf{r}_i$, every entry of $\mathbf{K}$ is given by the dot product of two columns of $\mathbf{R}$, i.e.,

$$K_{pq} = \mathbf{r}_p^\top \mathbf{r}_q = \sum_{m=1}^M R_{mp} R_{mq}. \quad (5.96)$$

When we now compute the partial derivative $\frac{\partial K_{pq}}{\partial R_{ij}}$ we obtain

$$\frac{\partial K_{pq}}{\partial R_{ij}} = \sum_{m=1}^M \frac{\partial}{\partial R_{ij}} R_{mp} R_{mq} = \partial_{pqij}, \quad (5.97)$$

Gradients of Vector-Valued Functions XIII

$$\partial_{pqij} = \begin{cases} R_{iq} & \text{if } j = p, p \neq q \\ R_{ip} & \text{if } j = q, p \neq q \\ 2R_{iq} & \text{if } j = p, p = q \\ 0 & \text{otherwise} \end{cases} . \quad (5.98)$$

From (5.94), we know that the desired gradient has the dimension $(N \times N) \times (M \times N)$, and every single entry of this tensor is given by ∂_{pqij} in (5.98), where $p, q, j = 1, \dots, N$ and $i = 1, \dots, M$.

Outline

Partial derivative

Gradients of Vector-Valued Functions

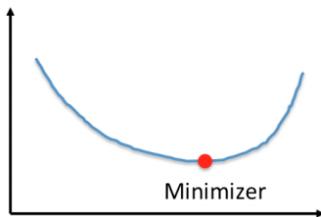
Gradient Descent and it's Variants

Gradient Descent I

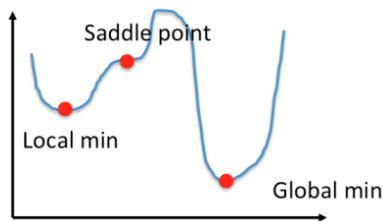
Convex and non-convex functions:

- ✓ A convex optimization problem is a problem where all of the constraints are convex functions, and the objective is a convex function if minimizing, or a concave function if maximizing.
- ✓ Linear functions are convex, so linear programming problems are convex problems.

Convex



Non-Convex

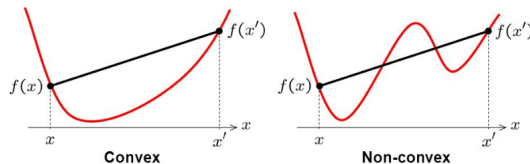


Gradient Descent II

A function $f : A \subseteq \mathbb{X} \rightarrow \mathbb{R}$ defined on a convex set A is called convex if

$$f(\lambda x + (1 - \lambda)x') \leq \lambda f(x) + (1 - \lambda)f(x') \quad (12)$$

for any $x, x' \in \mathbb{X}$ and $\lambda \in [0, 1]$.

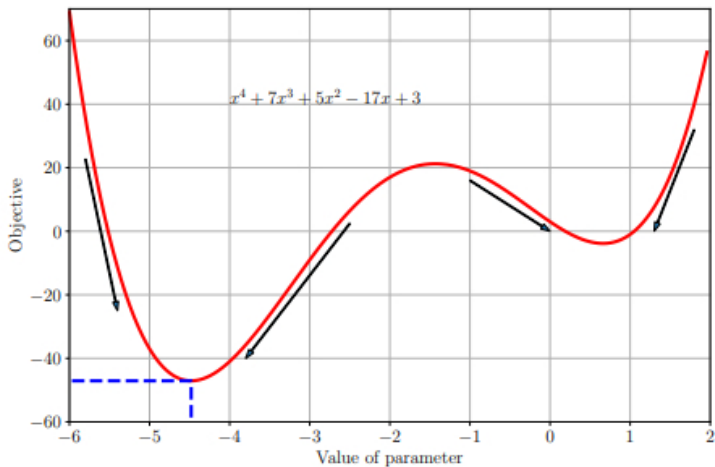


For convex function local minimum = global minimum

Gradient Descent III

- Gradient descent(GD) is the preferred way to optimize neural networks and many other machine learning algorithms but is often used as a black box.
- In a Neural Network, the Gradient Descent Algorithm is used during the forward and backward propagation to update the parameters of the model.
- Gradient descent will eventually converge to a stationary point of the function, regardless of convexity. If the function is convex, this will be a global minimum, but if not, it could be a local minimum or even a saddle point.

Gradient Descent IV



Gradient Descent V

- if f is convex there are no saddle points and all local minima are also global. Thus GD (**with a suitable stepsize**) is guaranteed to find a global minimizer.
- What makes non-convex optimization hard is the presence of saddle points and local minima, where the gradient is $(0, \dots, 0)$ and that have an arbitrarily bad objective value.
- Finding the global minimizer in such a setting is generally $\mathcal{NP}$ -hard and one instead settles with the goal of finding a local minimizer.
- The probability of GD to get stuck at a saddle is actually 0.
- However, the presence of saddle points might severely slow GDs progress down because directions of low curvature are exploited too slowly.

Gradient Descent VI

Problem of solving for the minimum of a real-valued function:

$$\min_x f(x), \quad (13)$$

where $f : \mathbb{R}^d \rightarrow \mathbb{R}$ is an objective function that captures the machine learning problem at hand. It is assumed that function is differentiable, and are unable to analytically find a solution in closed form.

Gradient descent is a first-order optimization algorithm

✓ Consider multivariate functions. Gradient descent exploits the fact that $f(x_0)$ decreases fastest if one moves from x_0 in the direction of the negative gradient $-((\nabla f)(x_0))^T$ of f at x_0 .

$$x_1 = x_0 - \gamma((\nabla f)(x_0))^T \quad (14)$$

for a small *step-size* $\gamma \geq 0$, then $f(x_1) \leq f(x_0)$.

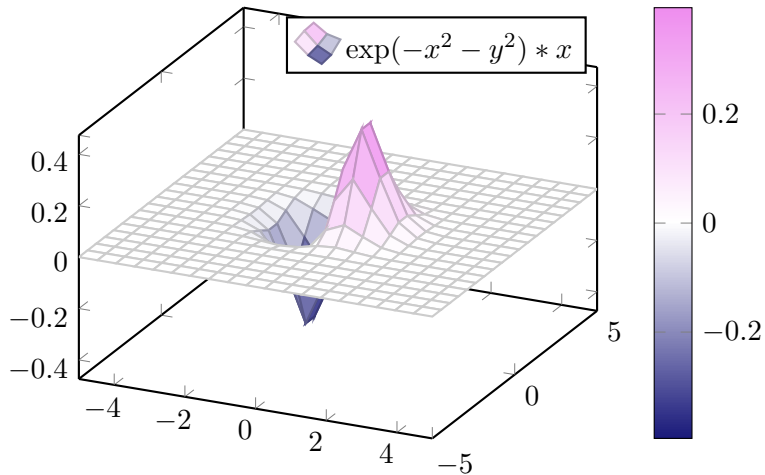
Gradient Descent VII

✓ To find a local optima $f(x_*)$ of a function $f : \mathbb{R}^n \rightarrow \mathbb{R}, x \mapsto f(x)$, start with an initial guess x_0 of the parameters and then iterate according to

$$x_{i+1} = x_i - \gamma_i((\nabla f)(x_i))^T \quad (15)$$

for suitable step-size γ_i , the sequence $f(x_0) \geq f(x_1) \geq \dots$ converge to a local minimum.

Gradient Descent VIII

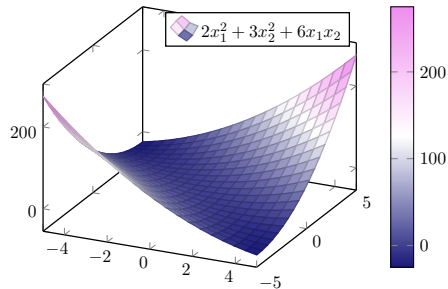


Gradient Descent IX

Example1: $2x_1^2 + 3x_2^2 + 6x_1x_2$

$$\equiv \begin{bmatrix} x_1 & x_2 \end{bmatrix} \begin{bmatrix} 2 & 3 \\ 3 & 3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \quad (16)$$

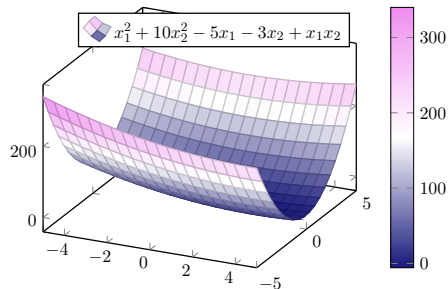
where $A = A'$



Gradient Descent X

Example2: Consider a quadratic function in two dimensions

$$f(x_1, x_2) = x_1^2 + 10x_2^2 - 5x_1 - 3x_2 + x_1x_2 \quad (17)$$



Gradient Descent XI

$$f\left(\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}\right) = \frac{1}{2} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}^T \begin{bmatrix} 2 & 1 \\ 1 & 20 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} - \begin{bmatrix} 5 \\ 3 \end{bmatrix}^T \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \quad (18)$$

with gradient

$$\nabla f\left(\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}\right) = \frac{1}{2} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}^T \begin{bmatrix} 2 & 1 \\ 1 & 20 \end{bmatrix} - \begin{bmatrix} 5 \\ 3 \end{bmatrix}^T \quad (19)$$

Starting at the initial location $x_0 = [-3, -1]^T$, [Equation \(15\)](#) iteratively applied to obtain a sequence of estimates that converge to the minimum value given in [Figure 1](#).

Gradient Descent XII

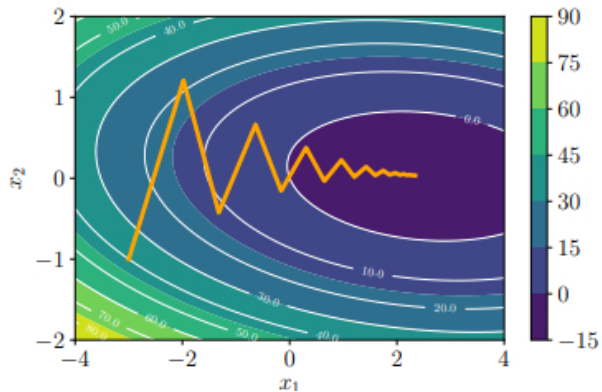


Figure: 1: Gradient descent on a two-dimensional quadratic surface

Gradient Descent XIII

Example3: Quadratic function minimization

$$f(x, y, z) = \frac{1}{2} \begin{bmatrix} x & y & z \end{bmatrix} \begin{bmatrix} 4 & 1 & 2 \\ 1 & 8 & 5 \\ 2 & 5 & 4 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} - \begin{bmatrix} x & y & z \end{bmatrix} \begin{bmatrix} 2 \\ 3 \\ 4 \end{bmatrix} = \frac{1}{2} \mathbf{x}^T A \mathbf{x} - \mathbf{x}^T b$$

$$f(x, y, z) = 2x^2 + 4y^2 + 2z^2 + xy + 5yz + 2zx - 2x - 3y - 4z$$

$$\begin{cases} f_x = 4x + y + 2z - 2 = 0 \\ f_y = x + 8y + 5z - 3 = 0 \\ f_z = 2x + 5y + 4z - 4 = 0 \end{cases}$$

$$\begin{bmatrix} 4 & 1 & 2 \\ 1 & 8 & 5 \\ 2 & 5 & 4 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} 2 \\ 3 \\ 4 \end{bmatrix}$$

Cost Function I

Cost Function II

Cost Function III

- ✓ It is a function that measures the performance of a model for any given data.
- ✓ Cost Function quantifies the error between predicted values and expected values and presents it in the form of a single real number.
- ✓ After making a hypothesis with initial parameters, Cost function is calculated.
- ✓ With a goal to reduce the cost function modify the parameters by using the Gradient descent algorithm over the given data (m , number of observations).

- **Hypothesis:** $h_{\theta}(x) = \theta_0 + \theta_1 x$
- **Parameters:** θ_0, θ_1
- **Cost Function:** $\mathcal{J}(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$
- **Gradient:** $\frac{\partial \mathcal{J}(\theta)}{\partial \theta_j} = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) \cdot X_j^{(i)}$
- **Goal:** $\min_{\theta_0, \theta_1} \mathcal{J}(\theta_0, \theta_1)$

Cost Function IV

$$\theta_0 := \theta_0 - \alpha \cdot \left(\frac{1}{m} \cdot \sum_{i=1}^m \left(h(\theta^{(i)} - y^{(i)}) \cdot X_0^{(i)} \right) \right) \quad (20)$$

$$\theta_1 := \theta_1 - \alpha \cdot \left(\frac{1}{m} \cdot \sum_{i=1}^m \left(h(\theta^{(i)} - y^{(i)}) \cdot X_1^{(i)} \right) \right) \quad (21)$$

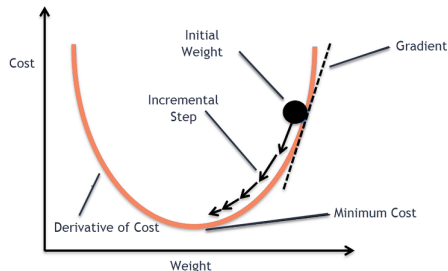
$$\theta_2 := \theta_2 - \alpha \cdot \left(\frac{1}{m} \cdot \sum_{i=1}^m \left(h(\theta^{(i)} - y^{(i)}) \cdot X_2^{(i)} \right) \right) \quad (22)$$

$$\vdots \quad (23)$$

$$\theta_j := \theta_j - \alpha \cdot \left(\frac{1}{m} \cdot \sum_{i=1}^m \left(h(\theta^{(i)} - y^{(i)}) \cdot X_j^{(i)} \right) \right) \quad (24)$$

Cost Function V

- ✓ Gradient descent is an iterative optimization algorithm for finding the local minimum of a function.
- ✓ To find the local minimum of a function using gradient descent, we must take steps proportional to the negative of the gradient (move away from the gradient) of the function at the current point.
- ✓ If we take steps proportional to the positive of the gradient (moving towards the gradient), we will approach a local maximum of the function, and the procedure is called Gradient Ascent.



Cost Function VI

- ✓ Gradient descent was originally proposed by CAUCHY in 1847. It is also known as steepest descent.
- ✓ The goal of the gradient descent algorithm is to minimize the given function (say cost function). To achieve this goal, it performs two steps iteratively:

- 1 Compute the gradient (slope), the first order derivative of the function at that point
- 2 Make a step (move) in the direction opposite to the gradient, opposite direction of slope increase from the current point by alpha times the gradient at that point.

Cost Function VII

Algorithm 1 Gradient descent algorithm

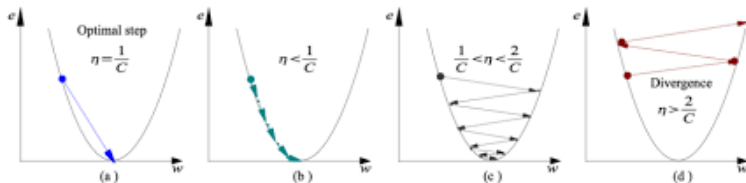
- 1: **while** (repeat until convergence) **do**
 - 2: $\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} \mathcal{J}(\theta_0, \theta_1);$
 - 3: **end while**
-

Alpha (α) : The learning rate

✓ After having the direction, we must decide the size of the step we must take.

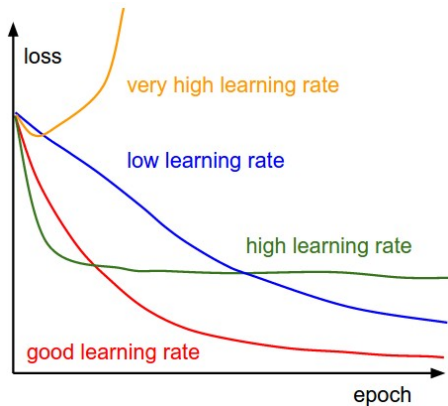
- Step-size must be chosen carefully to end up with local minima.
- If the learning rate is too high, we might OVERSHOOT the minima and keep bouncing, without reaching the minima
- If the learning rate is too small, the training might turn out to be too long

Cost Function VIII



- A** Learning rate is optimal, model converges to the minimum
- B** Learning rate is too small, it takes more time but converges to the minimum
- C** Learning rate is higher than the optimal value, it overshoots but converges ($1/C < \eta < 2/C$)
- D** Learning rate is very large, it overshoots and diverges, moves away from the minima, performance decreases on learning

Cost Function IX



As the gradient decreases while moving towards the local minima, the size of the step decreases. So, the learning rate (alpha) can be constant over the optimization and need not be varied iteratively.

Variants of Gradient Descent I

- ❶ **Batch Gradient Descent (BGD):** If all the observations in data set is used to calculate the cost function.
- ✓ Take the entire training set, perform forward propagation and calculate the cost function.
 - ✓ An epoch is when the entire training set is passed through the model, forward propagation and backward propagation are performed and the parameters are updated.
- One Epoch is when an ENTIRE dataset is passed forward and backward through the neural network only ONCE.**
- ✓ In batch Gradient Descent since the entire training set is used, the parameters will be updated only once per epoch.

Variants of Gradient Descent II

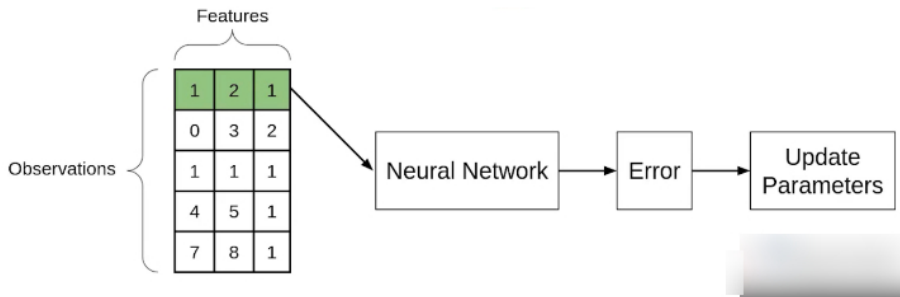
② **Stochastic Gradient Descent(SGD):** A single observation is used to calculate the cost function.

✓ Let's say we have 5 observations and each observation has three features and the values that have taken are completely random.

Features		
1	2	1
0	3	2
1	1	1
4	5	1
7	8	1

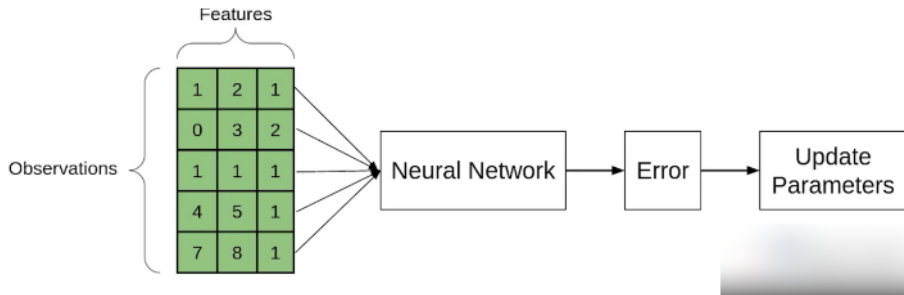
Variants of Gradient Descent III

- ✓ Take the first observation, then pass it through the neural network, calculate the error and then update the parameters.



- ✓ Then will take the second observation and perform similar steps with it. This step will be repeated until all observations have been passed through the network and the parameters have been updated.

Variants of Gradient Descent IV



- ✓ Each time the parameter is updated, it is known as an Iteration. Here since we have 5 observations, the parameters will be updated 5 times or we can say that there will be 5 iterations.
- ✓ In SGD, there will be 'm' iterations per epoch, where 'm' is the number of observations in a dataset.

Variants of Gradient Descent V

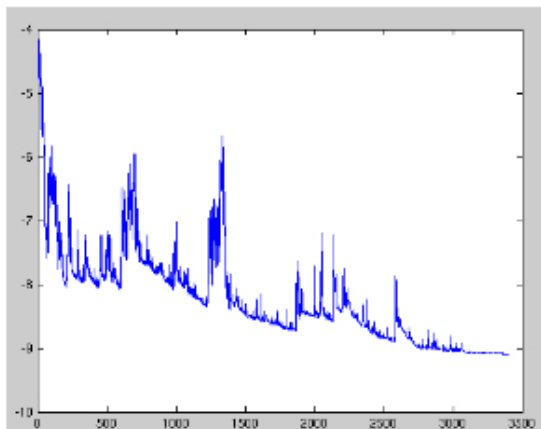


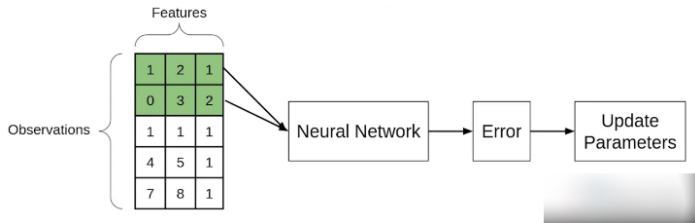
Figure: SGD fluctuation

Variants of Gradient Descent VI

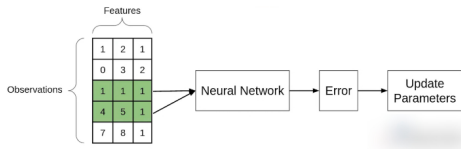
- ③ **Mini-batch Gradient Descent:** It takes a subset of the entire dataset to calculate the cost function. So if there are ‘m’ observations then the number of observations in each subset or mini-batches will be more than 1 and less than ‘m’.
- ✓ Assume that the batch size is 2. So we'll take the first two observations, pass them through the neural network, calculate the error and then update the parameters.
 - ✓ Mini-batch gradient descent finally takes the best of both worlds and performs an update for every mini-batch of n training examples:

$$\theta = \theta - \eta \cdot \nabla_{\theta} \mathcal{J} \left(\theta; x^{(i:i+n)}; y^{(i:i+n)} \right) \quad (25)$$

Variants of Gradient Descent VII

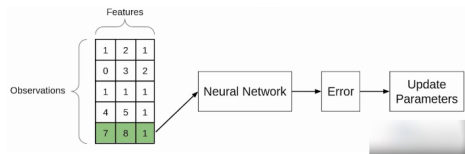


✓ Then take the next two observations and perform similar steps i.e will pass through the network, calculate the error and update the parameters.



Variants of Gradient Descent VIII

✓ Now since we are left with the single observation in the final iteration, there will be only a single observation and will update the parameters using this observation.



- Reduces the variance of the parameter updates, which can lead to more stable convergence;
- Can make use of highly optimized matrix optimizations common to state-of-the-art deep learning libraries that make computing the gradient w.r.t. a mini-batch very efficient.
- Common mini-batch sizes range between 50 and 256, but can vary for different applications.

Variants of Gradient Descent IX

- Mini-batch gradient descent is typically the algorithm of choice when training a neural network and the term SGD usually is employed also when mini-batches are used.

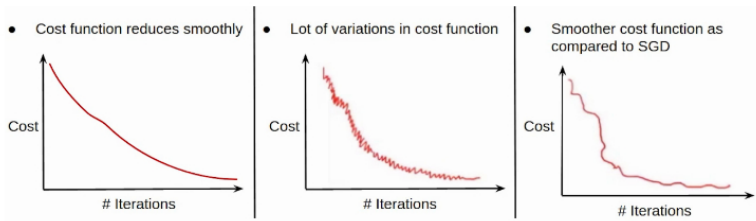
Variants of Gradient Descent X

Comparison between Batch GD, SGD, and Mini-batch GD:

✓ Comparison: Number of observations used for Update:

Batch Gradient Descent	Stochastic Gradient Descent (SGD)	Mini-Batch Gradient Descent
• Entire dataset for update	• Single observation for update	• Subset of data for update

Comparison: Cost function:



Variants of Gradient Descent XI

- Now, since we update the parameters using the entire data set in the case of the Batch GD, the cost function, in this case, reduces smoothly.
- On the other hand, this updation in the case of SGD is not that smooth. Since we're updating the parameters based on a single observation, there are a lot of iterations. It might also be possible that the model starts learning noise as well.
- The updation of the cost function in the case of Mini-batch Gradient Descent is smoother as compared to that of the cost function in SGD. Since we're not updating the parameters after every single observation but after every subset of the data.

Variants of Gradient Descent XII

Comparison: Computation Cost and Time

Batch Gradient Descent	Stochastic Gradient Descent (SGD)	Mini-Batch Gradient Descent
• Entire dataset for updation • Cost function reduces smoothly • Computation cost is very high	• Single observation for updation • Lot of variations in cost function • Computation time is more	• Subset of data for updation • Smoother cost function as compared to SGD • Computation time is lesser than SGD • Computation cost is lesser than Batch Gradient Descent

- Since it needs to load the entire data set at a time, perform the forward propagation on that and calculate the error and then update the parameters, the computation cost in the case of Batch gradient descent is very high.

Variants of Gradient Descent XIII

- Computation cost in the case of SGD is less as compared to the Batch Gradient Descent since it needs to load every single observation at a time but the Computation time here increases as there will be more number of updates which will result in more number of iterations.
- In the case of Mini-batch Gradient Descent, taking a subset of the data there are a lesser number of iterations or updations and hence the computation time in the case of mini-batch gradient descent is less than SGD.
 - ✓ Also, since it does not load the entire dataset at a time whereas loading a subset of the data, the computation cost is also less as compared to the Batch gradient descent.
 - ✓ **This is the reason Mini-batch gradient descent is preferred.**

Momentum I

- ✓ SGD has trouble navigating ravines, i.e. areas where the surface curves much more steeply in one dimension than in another, which are common around local optima.
- ✓ In these scenarios, SGD oscillates across the slopes of the ravine while only making hesitant progress along the bottom towards the local optimum.
- ✓ **Momentum** is a method that helps accelerate SGD in the relevant direction and dampens oscillations. It does this by adding a fraction γ of the update vector of the past time step to the current update vector:

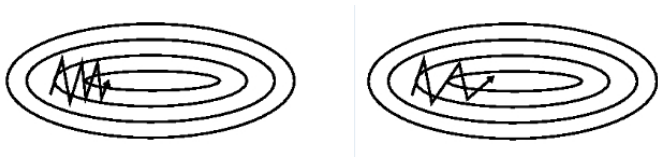


Figure: SGD without momentum and with momentum

Momentum II

$$v_t = \gamma v_{t-1} + \eta \nabla_{\theta} \mathcal{J}(\theta) \quad (26)$$

$$\theta = \theta - v_t \quad (27)$$

Different gradient descent algorithm I

- 1 Nesterov accelerated gradient: A limitation of gradient descent is that the progress of the search can slow down if the gradient becomes flat or large curvature. Momentum can be added to gradient descent that incorporates some inertia to updates. This can be further improved by incorporating the gradient of the projected new position rather than the current position, called Nesterov's Accelerated Gradient (NAG) or Nesterov momentum.
- 2 Adagrad(Adaptive Gradient Algorithm): It adapts the learning rate to the parameters, performing smaller updates (i.e. low learning rates) for parameters associated with frequently occurring features, and larger updates (i.e. high learning rates) for parameters associated with infrequent features. **For this reason, it is well-suited for dealing with sparse data.**
 - ✓ It greatly improved the robustness of SGD and used it for training large-scale neural nets at Google, which – among other things – learned to recognize cats in Youtube videos.¹

Different gradient descent algorithm II

- ③ Adadelta: Adadelta is a more robust extension of Adagrad that adapts learning rates based on a moving window of gradient updates, instead of accumulating all past gradients.
 - ✓ With Adadelta, we do not even need to set a default learning rate, as it has been eliminated from the update rule.
- ④ RMSprop(Root Mean Squared Propagation): is an extension of gradient descent and the AdaGrad version of gradient descent that uses a decaying average of partial gradients in the adaptation of the step size for each parameter.
- ⑤ Adam (Adaptive Moment Estimation): is another method that computes adaptive learning rates for each parameter. In addition to storing an exponentially decaying average of past squared gradients v_t like Adadelta and RMSprop, Adam also keeps an exponentially decaying average of past gradients m_t , similar to momentum.
 - ✓ Whereas momentum can be seen as a ball running down a slope, Adam behaves like a heavy ball with friction, which thus prefers flat minima in the error surface.

Different gradient descent algorithm III

Benefits of using Adam on non-convex optimization problems:

- Straightforward to implement.
- Computationally efficient.
- Little memory requirements.
- Invariant to diagonal rescale of the gradients.
- Well suited for problems that are large in terms of data and/or parameters.
- Appropriate for non-stationary objectives.
- Appropriate for problems with very noisy/or sparse gradients.
- Hyper-parameters have intuitive interpretation and typically require little tuning.

Parallelizing and distributing SGD:

- Hogwild!: A Lock-Free Approach to Parallelizing Stochastic Gradient Descent.

Different gradient descent algorithm IV

- Downpour SGD: Large Scale Distributed Deep Networks. It is an asynchronous stochastic gradient descent procedure which leverages adaptive learning rates and supports a large number of model replicas
- TensorFlow
- Elastic Averaging SGD: the communication and coordination of work among concurrent processes (local workers), is based on an elastic force which links the parameters they compute with a center variable stored by the parameter server (master). The algorithm enables the local workers to perform more exploration, i.e. the algorithm allows the local variables to fluctuate further from the center variable by reducing the amount of communication between local workers and the master.